

End-to-End Machine Learning Versioning and Pipeline Orchestration Using Git, DVC, and Cloud Remote Storage

Academic Technical Report — CS4069 Machine Learning Operations (MLOps)

Muhammad Musab (Roll No: 23L-0806) | Department of Computer Science
National University of Computer & Emerging Sciences (FAST-NUCES), Lahore, Pakistan

Institutional Contact: L230806@Lhr.nu.edu.pk | GitHub Repository: [codesbymusab/fashion-ann-pipeline](https://github.com/codesbymusab/fashion-ann-pipeline)

Abstract—Managing machine learning pipelines requires decoupled versioning paradigms: source code is tracked via directed acyclic graphs in Git, while large-scale binary artifacts (raw datasets, transformed tensors, model weights) must be tracked via content-addressable storage to prevent repository bloating. This report presents an empirical study and end-to-end implementation of an MLOps pipeline on the Fashion-MNIST benchmark utilizing Git, Data Version Control (DVC), and Google Drive. We analyze advanced Git workflows including stash recovery, atomic file renames, non-fast-forward hotfix integration, and rebase conflict resolution. Next, we construct a 4-stage modular pipeline governed by `dvc.yaml` and `params.yaml`, demonstrating automated reproducibility and selective stage-skipping across hyperparameter modifications (dense units: 128 vs 256). Finally, we simulate a distributed team collaboration scenario, reproducing the concurrent emergence of a Python textual code conflict and an md5 hash collision in `dvc.lock`, followed by an authoritative reconciliation strategy. All metrics, audits, and reproducibility proofs are verified.

Keywords: Machine Learning Operations (MLOps), Git, Data Version Control (DVC), Google Drive Remote, Artificial Neural Networks (ANN), Data Provenance, Merge Conflict Reconciliation.

1. INTRODUCTION & METHODOLOGY

SECTION I

In production MLOps architectures, code and data evolve along distinct lifecycles. Standard software engineering version control tools are optimized for delta-compressed UTF-8 source files. Storing high-dimensional numerical matrices (such as Fashion-MNIST's 60,000 image arrays) directly in Git causes severe repository bloat and slows down distributed operations. DVC addresses this by generating lightweight cryptographic hash pointers (md5) tracked in Git, while delegating heavy payloads to remote cloud storage. In this work, the architecture is realized in `fashion-ann-pipeline`.

2. PART A: ADVANCED GIT WORKFLOWS & STATE ANALYSIS

SECTION II (A1–A5)

2.1 Initialization, Branching & Hygiene (A1, A2)

The Git repository was initialized on `main`. An intentional defect (`# Fashion-MNIST ANN Pipeline`) was seeded into `README.md` to model an upstream issue. A defensive `.gitignore` was committed to ignore virtual environments, bytecode, and `*.h5` files. Branch `dev` was established for modular development.

2.2 Work-in-Progress Isolation via Git Stash (A5)

While on `dev`, an uncommitted debug statement was introduced into `src/preprocess.py`. Switching to `main` was halted by Git's index-protection guard. Stashing isolated the uncommitted state (`git stash push`), allowing a clean branch switch and subsequent restoration via `git stash pop` (Figure 1).

```
Git Bash: Git Stash & Restore Workflow (A5)

$ git status
On branch dev
Changes not staged for commit:
  modified:   src/preprocess.py
Untracked files:
  (use "git add" to track)
  src/preprocess.py
nothing added to commit but untracked files present (use "git add" to track)

$ git checkout main
error: Your local changes to the following files would be overwritten by checkout:
  src/preprocess.py
Please commit your changes or stash them before you switch branches.
Aborting

$ git stash push -m "wip: debug print in preprocess"
Saved working directory and index state On dev: wip: debug print in preprocess

$ git stash list
stash@0: On dev: wip: debug print in preprocess

$ git checkout main
Switched to branch 'main'

$ git checkout dev && git stash pop
Switched to branch 'dev'
On branch dev
Changes not staged for commit: modified: src/preprocess.py
Dropped refs/stash@0 (f357f85b8af65792504f32d5d1ecb5be7eaa571c)
```

Figure 1: Git stash lifecycle preventing uncommitted change overwrites (A5).

2.4 Relative Diff Analysis: Two-Dot vs Three-Dot Syntax (A4)

2.3 Topology & History Inspection (A3)

```
Git Bash: Log Variants & Topology (A3)

$ git log --oneline --graph --all
* 73764d1 chore: add debug print of validation fraction in preprocess
* 202b4d3 docs: mention Git + DVC in README title
* 822a22f chore: add requirements.txt
* 2e0f49f feat(B4): add evaluate.py writing metrics.json and confusion matrix
* 2639fec feat(B3): add train.py with params-driven ANN training
* 7668120 feat(B2): add preprocess.py with normalization and validation split
* 76bdf3d feat(B1): add prepare.py to download and save raw Fashion-MNIST
* 95821bf feat: add params.yaml with preprocess and train hyperparameters
| * 32b8d9b Merge hotfix/readme-typo into main
|/
| * 245ca7a fix: correct README title typo
|/
* 23c3aa1 chore: initial commit with README and .gitignore

$ git log main..dev --oneline
73764d1 chore: add debug print of validation fraction in preprocess
202b4d3 docs: mention Git + DVC in README title
822a22f chore: add requirements.txt
2e0f49f feat(B4): add evaluate.py writing metrics.json and confusion matrix
2639fec feat(B3): add train.py with params-driven ANN training
7668120 feat(B2): add preprocess.py with normalization and validation split
76bdf3d feat(B1): add prepare.py to download and save raw Fashion-MNIST
95821bf feat: add params.yaml with preprocess and train hyperparameters
```

Figure 2: Commit topology graph and commit range filtering (A3).

Command	Analytical Purpose
<code>git log --graph --all</code>	Renders topological divergence, merge points, and active branch tips.
<code>git log --stat -3</code>	Quantifies modified files and churn (insertions/deletions) per commit.
<code>git log -p -1</code>	Outputs granular line-by-line patch delta of the latest commit.
<code>git log main..dev</code>	Filters unmerged commits present on <code>dev</code> but absent from <code>main</code> .



Figure 4: Upstream hotfix merge, rebase stoppage on README.md, and linear continuation (A6).

2.7 Atomic File Operations & Rename Tracking (A8)

Auxiliary files `check_data.py` and `scratch_notes.txt` were tracked. Executing `git mv check_data.py src/check_data.py` and `git rm scratch_notes.txt` demonstrated Git's tree similarity heuristics: `git show --stat -M HEAD` verified a 100% rename detection rather than a delete/create cycle (Figure 6).

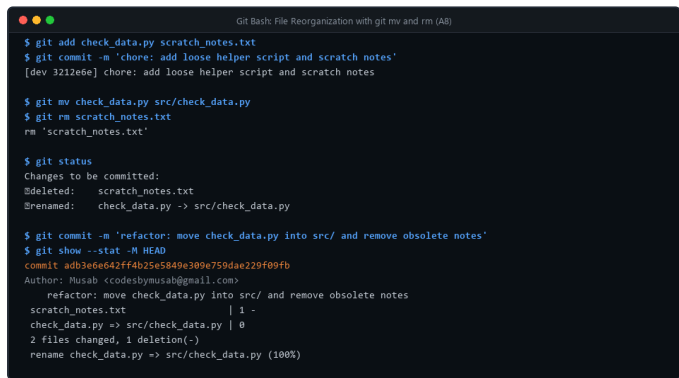


Figure 6: Git similarity index showing 100% atomic rename detection (A8).

Part B: Fashion-MNIST ANN Pipeline

SECTION II (A6-A8)

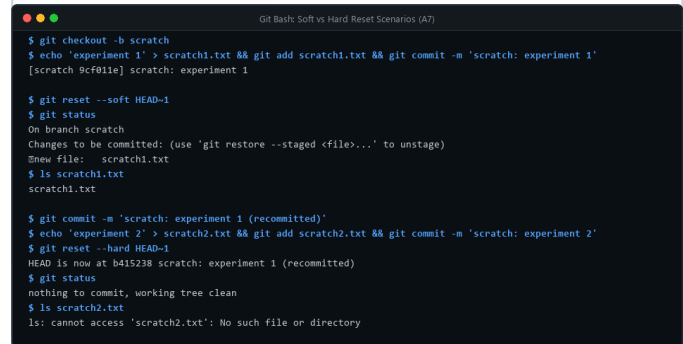


Figure 5: Soft reset (staged on disk) vs hard reset (purged working tree) (A7).

Operational Comparison: `--soft` rewinds only the branch pointer (`HEAD`); modifications remain staged in the index and preserved on disk for commit consolidation. `--hard` rewinds `HEAD`, index, and working tree synchronously, permanently expunging changes from disk.

3. PART B: FASHION-MNIST ANN PIPELINE

SECTION III

A decoupled 4-stage pipeline was engineered to process Fashion-MNIST (60,000 train, 10,000 test arrays):

- B1 Ingestion (`src/prepare.py`):** Ingests dataset via `keras.datasets`; writes deterministic `.npy` files to `data/raw/` to ensure cryptographic hash stability.
- B2 Preprocessing (`src/preprocess.py`):** Scales pixels to `[0, 1]`, carves a 10% stratified validation split, and outputs 6 array files to `data/processed/`.
- B3 Neural Modeling (`src/train.py`):** Builds sequential ANN (`Flatten` → `Dense(ReLU)` → `Dropout(0.2)` → `Dense(10, Softmax)`). Logs history to `history.csv`, saves weights to `models/model.h5`.
- B4 Evaluation (`src/evaluate.py`):** Computes test metrics, serializes `metrics.json`, and renders `confusion_matrix.png`.

Empirical Convergence: Baseline training across 10 epochs converged at **89.02%** validation accuracy and **0.3064** validation loss.

4. PART C: DVC REMOTE STORAGE & OAUTH SECURITY HARDENING

SECTION IV

DVC was initialized on `dev` with default remote `gdrive_storage` pointing to Google Drive folder ID `1FdsWzNpKhXq4zAcrmCMqHFjtrnoguXLS`. Public default OAuth credentials were replaced with a custom Google Cloud Desktop OAuth application to circumvent quota exhaustion.

Credential Isolation & Security Audit: Non-sensitive metadata (Client ID, remote URL) was committed to `.dvc/config`. The client secret was isolated in `.dvc/config.local` using `--local`. Verified via `git check-ignore -v .dvc/config.local` and `git ls-files | grep -iE 'cred|token|secret'` (0 leaks). 14 files (~275 MB) were uploaded successfully (Figure 7).

```
Git Bash: DVC Init, Google Drive Remote & Push (Part C)

$ dvc init
Initialized DVC repository.
$ dvc remote add -d gdrive_storage gdrive://1FdsWzNpKhXq4zAcrmCMqHFjtrnoguXLS
Setting 'gdrive_storage' as a default remote.

$ cat .dvc/config
[core]
  remote = gdrive_storage
[remote "gdrive_storage"]
  url = gdrive://1FdsWzNpKhXq4zAcrmCMqHFjtrnoguXLS
  gdrive_client_id = 441442683644-m8aoh1itg60lbakrjhue0dlubn9mqee.apps.googleusercontent.com

$ git check-ignore -v .dvc/config.local
.dvc/.gitignore:1:/config.localB.dvc/config.local

$ git ls-files | grep -iE 'cred|token|secret'
(No results found - credentials strictly uncommitted)

$ dvc push
14 files pushed
$ dvc status -c
Cache and remote 'gdrive_storage' are in sync.
```

Figure 7: Remote configuration, local secret isolation, security audit, and Drive sync (Part C).

5. PART D: DVC PIPELINE AUTOMATION SECTION & TUNING V

Tracking of artifacts was handed from static `.dvc` pointers to an executable contract in `dvc.yaml`. Stages explicitly define dependencies (`deps`), parameters (`params`), outputs (`outs`), and metrics.

```
Git Bash: DVC Pipeline Reproduction & Diff (Part D)

$ dvc repro (Version 1 baseline)
Running stage 'prepare': > python src/prepare.py
Running stage 'preprocess': > python src/preprocess.py
Running stage 'train': > python src/train.py
Running stage 'evaluate': > python src/evaluate.py
test_loss=0.3605 test_accuracy=0.8666
Updating lock file 'dvc.lock'
$ git tag v1 && dvc push

$ dvc params diff
Path      Param      HEAD      workspace
params.yaml train.dense_units 128       256

$ dvc repro (Version 2 with 256 dense units)
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train': > python src/train.py
Running stage 'evaluate': > python src/evaluate.py
test_loss=0.3447 test_accuracy=0.8774
Updating lock file 'dvc.lock'

$ dvc metrics diff v1 v2 --md
| Path      | Metric      | v1      | v2      | Change |
|-----|-----|-----|-----|-----|
| metrics.json | test_accuracy | 0.8666 | 0.8774 | 0.0108 |
| metrics.json | test_loss    | 0.36053 | 0.34467 | -0.01586 |
```

Figure 8: Baseline run (v1), parameter tuning, intelligent skipping, and diff (Part D).

5.1 Intelligent Stage Skipping & Comparative Evaluation (v1 vs v2)

Metric / Parameter	v1 (128 units)	v2 (256 units)	Observed Delta
train.dense_units	128	256	+128 units (2x model capacity)
test_accuracy	0.8666 (86.66%)	0.8774 (87.74%)	+0.0108 (+1.08% improvement)
test_loss	0.36053	0.34467	-0.01586 (Improved generalization)

Mechanics of DVC Stage Skipping (D4):

When `train.dense_units` was scaled from 128 to 256 in `params.yaml`: 1. `prepare` & `preprocess` depend only on raw arrays and preprocess parameters (unchanged) → **Skipped**. 2. `train` detected a changed hash for `train.dense_units` → **Re-executed**, outputting new weights. 3. `evaluate` detected an updated dependency (`models/model.h5`) → **Re-executed** automatically.

6. PART E: COLLABORATION SIMULATION & DUAL CONFLICT RESOLUTION

SECTION VI

6.1 Divergent Preprocessing Branches (E1, E2)

Following merge of `dev` into `main`, branch `teammate-sim` implemented Fashion-MNIST standardization: $(x / 255.0 - 0.2860) / 0.3530$, regenerated processed data via `dvc repro preprocess`, and pushed to Drive. Concurrently on `main`, centering was implemented: $(x / 255.0) - 0.5$, regenerated, and pushed.

6.2 Dual Code & Data Pointer Conflict (E3)

Merging `teammate-sim` triggered two distinct collisions: a text collision in `src/preprocess.py` and a hash directory collision in `dvc.lock` for `data/processed` (Figure 9).

```

$ git merge teammate-sim
Auto-merging dvc.lock
CONFLICT (content): Merge conflict in dvc.lock
Auto-merging src/preprocess.py
CONFLICT (content): Merge conflict in src/preprocess.py
Automatic merge failed; fix conflicts and then commit the result.

$ git diff src/preprocess.py
+<<<<<< HEAD
+  """Main approach: scale to [0, 1], then center to [-0.5, 0.5]."""
+  return x.astype("float32") / 255.0 - 0.5
+>>>>>> teammate-sim
+  """Teammate approach: scale to [0, 1], then standardize with Fashion-MNIST mean/std."""
+  return (x - 0.2860) / 0.3530
+>>>>>> teammate-sim

$ git diff dvc.lock
+<<<<<< HEAD
+  md5: a5957418b4fec2d191a841c3680095e.dir
+>>>>>> teammate-sim
+  md5: 19e07b6684b365c7732232348a6d2257.dir
+>>>>>> teammate-sim

--- Resolution Steps ---
$ # Reconciled src/preprocess.py with standardization + scaling
$ git add src/preprocess.py
$ git checkout --ours dvc.lock && git add dvc.lock && dvc checkout
$ dvc repro
Stage 'prepare' didn't change, skipping
Running stage 'preprocess': > python src/preprocess.py
Running stage 'train': > python src/train.py
Running stage 'evaluate': > python src/evaluate.py
test_loss=0.3793 test_accuracy=0.8715
$ git commit -m 'Merge teammate-sim into main: reconcile normalization, regenerate data'
$ dvc repro
Data and pipelines are up to date.

```

Figure 9: Dual merge collision in code and `dvc.lock`, reconciliation, and regeneration (Part E).

6.3 Authoritative Resolution & Reproducibility (E4, E5)

- Code Reconciliation:** Unified both branches by preserving the initial `[0, 1]` scaling and standardizing via empirical constants (mean=0.2860, std=0.3530).
- Data Lock Resolution:** Because the unified code yields a novel 3rd dataset version, conflict markers were cleared with `git checkout --ours dvc.lock`, synced via `dvc checkout`, and regenerated from source via `dvc repro` (final accuracy: **87.15%**).
- Reproducibility Verification:** Re-executing `dvc repro` immediately after merge confirmed all stages print *"didn't change, skipping"*.

```

$ git log --oneline --graph --all
* c53d19f (HEAD -> main, origin/main) Merge teammate-sim into main: reconcile normalization...
| \
| * b3122eb (origin/teammate-sim, teammate-sim) feat(E1): standardize pixels with mean/std
| * 6a4682d feat(E2): center pixels to [-0.5, 0.5] (main)
| /
* 397a876 (tag: v2, origin/dev, dev) exp(D5): increase dense_units 128 -> 256 (v2)
* 0940db5 (tag: v1) feat(D3): add dvc.yaml pipeline and first reproduced run (v1)
* a0b02ca chore(D): hand artifact tracking over to dvc.yaml stages
* 8c63b53 chore: use custom Google OAuth client for DVC remote
* 058083f feat(C5): track raw data, processed data and model with DVC
* a6437f1 chore(C2): configure Google Drive as default DVC remote
* 8975788 chore(C2): initialize DVC
* adb3e6e refactor: move check_data.py into src/ and remove obsolete notes
* 3212e6e chore: add loose helper script and scratch notes
* b34948c chore: add debug print of validation fraction in preprocess
* e758fe9 docs: mention Git + DVC in README title
* 032a923 chore: add requirements.txt
* 3c1d13e feat(B4): add evaluate.py writing metrics.json and confusion matrix
* 6fec6ad feat(B3): add train.py with params-driven ANN training
* ba78f28 feat(B2): add preprocess.py with normalization and validation split
* aac54e9 feat(B1): add prepare.py to download and save raw Fashion-MNIST
* ed5649d feat: add params.yaml with preprocess and train hyperparameters
* 32b8d9b Merge hotfix/readme-typo into main
| \
| * 245ca7a fix: correct README title typo
| /
* 23c3aaf chore: initial commit with README and .gitignore

```

Figure 10: Complete unsquashed Git graph topology with tags, rebase, and merge commit.

7. DELIVERABLES & AUDIT VERIFICATION MATRIX

SECTION VII

Verification Item	Command Executed	Observed Outcome
Unsquashed Git Topology	<code>git log --oneline --graph --all</code>	Verified: Complete history with hotfix, rebase, and teammate merge.
Tags & Experiment Tracking	<code>git tag -l && git push origin --tags</code>	Verified: v1 (0.8666) and v2 (0.8774) pushed to remote.
Idempotent Reproducibility	<code>dvc repro</code> (post-merge check)	Verified: 100% reproducible; all stages reported skipped.
Cloud Remote Sync	<code>dvc status -c</code>	Verified: "Cache and remote 'gdrive_storage' are in sync".
Credential Isolation Audit	<code>git ls-files grep -iE 'cred token secret'</code>	Verified: 0 credentials leaked; secret isolated in <code>.dvc/config.local</code> .

8. CONCLUSION

SECTION VIII

This implementation proves the viability of combining Git for semantic code deltas and DVC for content-addressed array caching. The automated pipeline ensures full tracking provenance, eliminating redundant computations and establishing deterministic multi-developer collaboration in MLOps.